AI 454: NLP

Implementation assignment (mini project) 2

Due: May 1, 2026, at 11:59 PM sharp

General instructions

Language: implement your assignment in **Pytorch**

Teams of Two (Mandatory): This assignment must be completed in teams of exactly two people.

Collaboration Requirement: Teams are expected to discuss every detail together, rather than dividing the workload. Both team members should be actively involved in all aspects of the assignment, from data processing and model implementation to reporting results.

Code: Make it self-contained and include clear run instructions. Submit via e-learning, with team members' names in the code and report.

Documentation: **DON'T add documentation to your code** at all (should be free of documentation)

Content: Answer all questions in the report.

Grade: This assignment is graded out of **80 points (70 +10bonus)** and contributes **15% of the overall course grade (+1 bonus)**

We will consider the code, report, and, importantly, the quality of group discussion. Missing discussion on any required point will result in losing weight for that point (this includes the ability to answer any question related to the code). If a group fails to discuss three or more points, they risk losing the entire assignment grade —so put in your best effort!

Plagiarism Warning: You may ask AI tools general programming questions, but **not to generate task-specific code, analogies, tables, or discussions**.

This assignment requires time, so do not leave it to the last few days.

.

Task Description

The dataset used in this assignment is a **sentiment classification** corpus consisting of short English-language sentences, each labeled with one of **three sentiment classes**.

Each example in the dataset has two columns:

- sentence: a single user-generated sentence
- class: the sentiment label, with the following mapping:
    - negative
    - neutral
    - positive

The dataset has been randomly shuffled and split into three sets: a **training set** for model development, a **development (validation) set** for tuning and model selection, and a **test set** for final evaluation.

You should use:
- only the **training set** for fitting the model parameters
- the **dev set** for tuning hyperparameters and selecting the best configuration
- the **test set** strictly for reporting results of the chosen models

General Evaluation Requirements (applies to all parts)

For **every model** and for **both dev and test sets**, you must report:

- **Accuracy**
- **Per-class Precision**, **Recall**, and **F1-score** for each class: **negative, neutral, positive**
- **Macro-averaged Precision, Recall, and F1-score**
- The full **confusion matrix**

Present your results clearly in your report using **well-organized tables** and label all figures or plots.

Part 1: LSTM vs BiLSTM (15 points)

In this part, you will implement and compare two recurrent neural network models for **sentiment classification**:

1. **Unidirectional LSTM** – A standard LSTM that processes input sequences from left to right.

2. **Bidirectional LSTM (BiLSTM)** – A model that processes input in both directions, allowing the network to leverage both past and future context.

You are required to use **pre-trained GloVe embeddings** (e.g., glove.6B.100d.txt) to initialize your embedding layer. Handle out-of-vocabulary words using appropriate strategies (e.g., a special <UNK> token or random vectors).

You should explore and tune hyperparameters including hidden layer size, dropout rate, optimizer and learning rate, batch size, and number of epochs.

Expected Deliverables (Part 1)

1. **Two PyTorch implementations**:
    - LSTM classifier
    - BiLSTM classifier
      both using GloVe embeddings.
2. A **hyperparameter tuning summary**, including:
    - The list of hyperparameters you tuned
    - The range of values explored for each
    - The best combination of hyperparameters found for each model
3. A **thorough evaluation and comparison**, including the requested evaluation metrics (i.e., accuracy per-class precision, etc.) reported on both the **development and test sets**:
4. A **comparative analysis** discussing:
    - Which model performed better and why.
    - Trade-offs between performance, complexity, and training time
    - Insights from the confusion matrix (e.g., which sentiment types are most often confused)
    - Any notable patterns in per-class performance (e.g., rare classes vs frequent ones)

Note (for all parts): It is up to you to decide which tables and visualizations to include in the report—at this point, you should be able to identify what is needed to support your analysis effectively.

Part 2: BiLSTM vs BiRNN (10 points)

In this part, you will compare the performance of a **Bidirectional LSTM (BiLSTM)** with a **Bidirectional RNN (BiRNN)** that uses vanilla RNN cells. The goal is to assess whether LSTM's gating mechanisms lead to better results than standard RNNs for sentiment classification.

You are expected to **reuse your best BiLSTM implementation from Part 1 without modification** (same architecture and hyperparameters that you selected as "best" in Part 1). For the BiRNN model:

- Use the **same hidden layer dimension, optimizer type, and batch size** as in the BiLSTM to ensure a fair comparison.
- You may tune other hyperparameters for BiRNN, including hidden layer size, dropout rate, optimizer and learning rate, batch size, and number of epochs.
- Both models should still use **pre-trained GloVe embeddings**.

Expected Deliverables (Part 2)

1. **PyTorch implementation** of the **BiRNN** model initialized with GloVe.
2. A **summary of the hyperparameters** you tuned specifically for BiRNN.
3. A **thorough evaluation and comparison** of BiLSTM vs BiRNN, including the metrics (for both dev and test sets):
4. A **comparative analysis** discussing:
    - Performance differences between BiLSTM and BiRNN
    - Trade-offs in terms of accuracy, model simplicity, and training time
    - Common limitations of vanilla RNNs (e.g., vanishing gradients, difficulty with long-range dependencies)
    - Key insights from the confusion matrices and class-level metrics

Part 3: Effect of Additive Attention on BiLSTM (10 points)

In this part, you will evaluate whether **attention pooling** improves performance in sentiment classification by comparing:

1. **BiLSTM without attention** (your best model from Part 1)
2. **BiLSTM with additive attention**, where a feedforward scoring function is used to compute attention weights over the BiLSTM hidden states.

You must **reuse the same BiLSTM configuration from Part 1** (e.g., hidden size, dropout, optimizer). You may **only adjust the learning rate and number of training epochs** for the attention-based model.

Attention Mechanism Description

Let h_t be the BiLSTM hidden state at time step t, formed by concatenating the forward and backward LSTM states:

$$h_t = [\overrightarrow{h_t}; \overleftarrow{h_t}]$$

The **attention score** for each time step is computed as:

$$e_t = v^\top \tanh\,(W h_t)$$

where W and v are learnable parameters. These scores are normalized using a softmax to obtain attention weights:

$$\alpha_t = \frac{\exp\,(e_t)}{\sum_k \exp\,(e_k)}$$

The final **sentence representation** is a weighted sum of hidden states:

$$c = \sum_t \alpha_t h_t$$

This context vector c is then passed to a classifier to predict the sentiment class.

Expected Deliverables (Part 3)

1. A **PyTorch implementation** of **BiLSTM with additive attention** (using the same BiLSTM backbone as in Part 1).
2. A **hyperparameter tuning summary** for the attention-based model:
    - Please include only the parameters you tuned **for this experiment** (e.g., learning rate, number of epochs, attention hidden size), not the ones inherited from previous parts (for these, use the best of what you get in the previous parts).
3. A **thorough evaluation and comparison**, with metrics reported on both dev and test sets:
4. **Attention visualization** for:

- **One correctly classified example**, and
- **One incorrectly classified example**.

Display the sentence along with the corresponding attention weights, for example:

- As a heatmap over tokens, or
- As color-coded text where intensity reflects the attention weight.

5. A **comparative analysis** discussing:
   - Whether attention improved performance and why (or why not)
   - Which tokens the model tends to focus on for different sentiment classes
   - How attention helps (or fails) in interpretability
   - Any notable patterns you observe in correct vs incorrect examples

Note: Use the same BiLSTM backbone for both models. Clearly present results and visualizations in your report and document your code for reproducibility.

Part 4: Self-Attention Transformer Encoder with Learned Positional Encodings (20 points)

In this part, you will implement a **self-attention–based Transformer encoder** for sentiment classification and compare it with your best BiLSTM-based model.

Unlike recurrent models, Transformer encoders use **multi-head self-attention** without recurrence. They require **positional encodings** to represent token order.

Input Representation and Positional Encodings

For this part, you must explicitly:

- Fix a **maximum sequence length** L(e.g., $L = 50$). All input sentences must be tokenized and either truncated or padded to length L.
- Use an **embedding dimension** d(e.g., $d = 100$, matching GloVe 100d).

The Transformer input will have shape:

- **Token embedding**: X ∈ ℝ^{B × L × d}, where:
  - B= batch size
  - L= fixed sequence length
  - d= embedding dimension

You must use **learned positional encodings**:

- Define a **learnable positional embedding matrix** $P \in \mathrm{R}^{L \times d}$.
- For a batch of token embeddings X, you construct:

$$Z = X + P$$

where for each position t, the same positional vector $P_t$is added across the batch. The resulting tensor Zhas shape $B \times L \times d$and is fed into the Transformer encoder layers.

Model Requirements

You must implement a **minimal Transformer Encoder classifier** that includes:

- Token embeddings (initialized randomly or with GloVe; your choice)
- **Learned positional embeddings** of size $L \times d$, added elementwise to token embeddings
- At least **one self-attention encoder layer**, built from:
    - Multi-head self-attention (nn.MultiheadAttention or equivalent) working on inputs of size $B \times L \times d$
    - A position-wise feed-forward network (two-layer MLP with a nonlinearity)
    - Residual connections and layer normalization
- A **sentence-level pooling operation**, such as:
    - Mean pooling over all non-padding positions, or
    - A special [CLS]-like token if you choose to add one
- A final **classification layer** mapping from d(or pooled dimension) to the number of sentiment classes.

Important: You must implement your own small Transformer encoder using PyTorch building blocks. Pre-trained models such as BERT, RoBERTa, or DistilBERT are **not allowed**.

Constraints

- Reuse the **same vocabulary, tokenization, and preprocessing** as in earlier parts.
- Use the **same embedding dimension** $d$as in your BiLSTM model (e.g., 100).
- The **positional embedding matrix** must be learnable and of size $L \times d$, where $L$is the maximum sequence length you choose.

- You may tune: Number of attention heads, Dropout rate, Learning rate Number of epochs, and Feed-forward layer size

All design choices must be documented clearly.

Expected Deliverables (Part 4)

1. A **PyTorch implementation** of the **Transformer Encoder classifier** with **learned positional encodings**, including clear documentation of:
    - The maximum sequence length Lused
    - The embedding dimension d
    - The size of the positional embedding matrix $L \times d$
2. **Training and evaluation results** on dev and test sets:
3. A **quantitative comparison** with your best BiLSTM-based model (from Parts 1–3), including:
    - Final metrics (accuracy, macro-Precision/Recall/F1)
    - Number of trainable parameters
    - Average training time per epoch
4. A **qualitative analysis** discussing:
    - Whether the Transformer outperformed the BiLSTM-based models, and why
    - How training time and parameter count compare
    - Whether self-attention seems to help with particular sentiment classes or longer sentences

---

Part 5: Final Summary and Comparative Analysis (15 points)

In this final part, you will **summarize and compare all models**:

- LSTM (Part 1)
- BiLSTM (Part 1)
- BiRNN (Part 2)
- BiLSTM with additive attention (Part 3)
- Transformer encoder with learned positional encodings (Part 4)

Required Summary Elements

You must include:

1. A **summary table** comparing all five models on both dev and test sets.
2. A **single figure** that displays confusion matrices for all five models in a clearly labeled layout (e.g., a grid of subplots).

Results Summary Table

For each model, report at least:

- Dev accuracy and test accuracy
- Dev and test **macro Precision, macro Recall, macro F1-score**
- Optionally, per-class F1 scores (you may include all three classes or highlight those most relevant to your discussion)
- **Number of trainable parameters**
- **Average training time per epoch**

You may also add other indicators (e.g., convergence speed, stability) if helpful.

Written Comparative Analysis

Using your summary table and confusion-matrix figure, provide a detailed, well-structured discussion. Address:

- How did different architectures affect classification performance overall?
- Did bidirectionality help compared to unidirectional LSTM?
- Did additive attention improve performance and/or interpretability?
- How did the self-attention Transformer compare to the BiLSTM-based models—in terms of:
    - Accuracy and macro F1
    - Behavior on minority classes
    - Parameter count and training time
- Which model do you consider **best overall**, and according to which criteria (accuracy, macro F1, interpretability, efficiency, etc.)?
- Were there any **surprising patterns or failures**, such as:
    - Certain sentiment classes consistently being confused
    - Models that overfit or underfit more clearly

- Architectures that helped with rare classes more than others

---

Report and Code Quality (10 points)

- Code quality (structure only — **documentation is not required**).

- The **quality and organization** of your report are worth **10 points** and will be graded across the entire assignment. This includes:

- Clear structure (sections with headings, consistent ordering of parts)
- Well-labeled tables and plots
- Clear explanations of experimental choices and hyperparameter decisions
- Correct and consistent notation
- Good writing quality and formatting

A neat, well-organized, and well-explained report that is easy to follow will be rewarded.